

Simulation-Based Inference

Exercise 3

Šimon Kucharský, Daniel Habermann

First we read the data (the data set is also provided in the `pscl` R package).

```
library(pscl)
data("bioChemists")
N <- nrow(bioChemists)
set.seed(582)
```

Solution for a) and b)

First, we fit a Poisson regression model on the original data, store the estimated coefficients for later and also estimate the overdispersion statistic $\hat{\phi}$.

```
fit_poisson <- glm(art ~ phd + ment, family = "poisson", data = bioChemists)

b0 <- coefficients(fit_poisson)["(Intercept)"]
b1 <- coefficients(fit_poisson)["phd"]
b2 <- coefficients(fit_poisson)["ment"]
phi <- sum(
  residuals(fit_poisson, type="response")^2 /
  predict(fit_poisson, type="response")
) / df.residual(fit_poisson)
```

To conduct a simulation-based test of $H_0 : b_1 = 0$ vs. $H_1 : b_1 \neq 0$, we will need to repeatedly sample values of the response variable from the null model

$$\begin{aligned} \text{art}_i^{(s)} &\sim \text{Poisson}(\lambda_i) \\ \lambda_i &= \exp(\hat{b}_0 + \hat{b}_2 \text{ment}_i), \end{aligned}$$

where we set the nuisance parameters b_0 and b_1 to their estimated values from the original data.

For each simulation $s \in \{1, \dots, S\}$, we fit the poisson regression model, and save the estimated parameter value $\hat{b}_1^{(s)}$. This will be our null distribution. We do the same for the dispersion statistic ϕ in question b), and so we can obtain both null distributions within one simulation loop.

```
lambda <- exp(b0 + b2 * bioChemists[["ment"]])

S <- 1e3
b1_sim <- numeric(S)
phi_sim <- numeric(S)
for(s in 1:S) {
  art_sim <- rpois(N, lambda)
```

```

data_sim <- data.frame(
  art  = art_sim,
  phd  = bioChemists[["phd"]],
  ment = bioChemists[["ment"]]
)
fit_sim <- glm(art ~ phd + ment, family = "poisson", data = data_sim)
b1_sim[s] <- coefficients(fit_sim)["phd"]
phi_sim[s] <- sum(
  residuals(fit_sim, type="response")^2 / predict(fit_sim, type="response")
) / df.residual(fit_sim)
}

```

a) Hypothesis test $H_0 : b_1 = 0$ vs. $H_1 : b_1 \neq 0$

Obtaining the simulation-based p -value of b_1 , we calculate the relative frequency with which the samples of $|\hat{b}_1^{(s)}|$ exceed the value of the observed coefficient $\hat{b}_1$.

```
(p_b1 <- mean(abs(b1_sim) >= abs(b1)))
```

```
[1] 0.659
```

The coefficient b_1 is not statistically significant.

b) Hypothesis test $H_0 : \phi = 1$ vs. $H_1 : \phi > 1$

The same is done for the dispersion statistic ϕ , with the only difference that we only compute a one-sided p -value.

```
(p_phi <- mean(phi_sim > phi))
```

```
[1] 0
```

The dispersion statistic is highly significant, as we practically have not seen any simulation from the Poisson model that would yield a larger dispersion than what was observed in the data. This indicates that the data are overdispersed.

Solution for c)

To conduct a simulation-based test using the negative binomial regression, we follow the same pattern as in question a). Note that the negative binomial regression estimates additional dispersion parameter θ , which needs to be set to its estimated value during simulation as it is a nuisance parameter.

```
fit_nb <- MASS::glm.nb(art ~ phd + ment, data = bioChemists)
b0 <- coefficients(fit_nb)["(Intercept)"]
b1 <- coefficients(fit_nb)["phd"]
b2 <- coefficients(fit_nb)["ment"]
theta <- summary(fit_nb)[["theta"]]

mu <- exp(b0 + b2 * bioChemists[["ment"]])
S <- 1e3
b1_sim <- numeric(S)
for(s in 1:S) {
  art_sim <- rnbinom(N, theta, mu=mu)
  data_sim <- data.frame(
    art = art_sim,
    phd = bioChemists[["phd"]],
    ment = bioChemists[["ment"]]
  )
  fit_sim <- MASS::glm.nb(art ~ phd + ment, data = data_sim)
  b1_sim[s] <- coefficients(fit_sim)["phd"]
}

(p_b1 <- mean(abs(b1_sim) >= abs(b1)))
```

```
[1] 0.741
```

The coefficient b_1 is not statistically significant under the negative binomial model either.

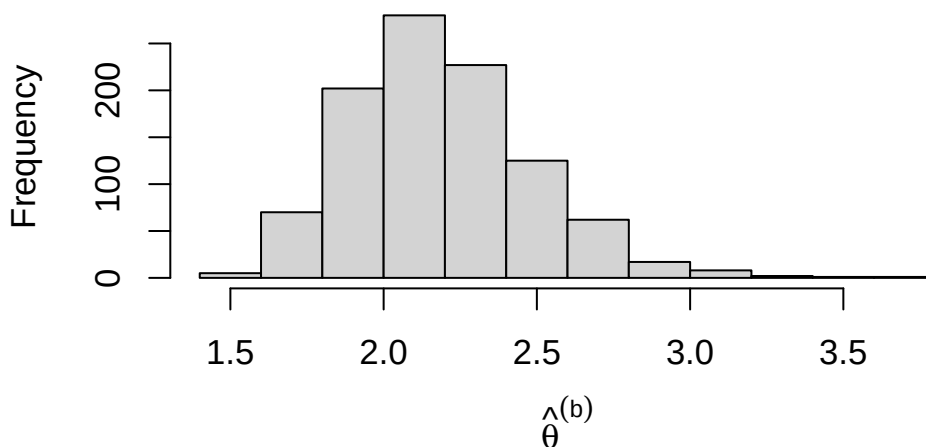
Solution for d)

To obtain a bootstrap distribution of the dispersion parameter, we will repeatedly resample the observed data with replacement, and fit the negative binomial regression. Then, we store the estimated values of $\hat{\theta}^{(b)}$

```
B <- 1000
theta_boot <- numeric(B)
for(b in 1:B) {
  rows <- sample.int(nrow(bioChemists), replace=TRUE)
  data_boot <- bioChemists[rows,]

  fit_boot <- MASS::glm.nb(art ~ phd + ment, data = data_boot)
  theta_boot[b] <- fit_boot[["theta"]]
}
```

```
par(cex=1.1)
hist(
  x=theta_boot,
  main="Histogram of bootstrapped dispersion",
  xlab=expression(hat(theta)^(b))
)
```

Histogram of bootstrapped dispersion

Calculating the BPI is equivalent to calculating the quantiles from the set of bootstrapped replicates $\{\hat{\theta}^{(b)}\}_{b=1}^B$: $[Q_{0.025}^{(B)}, Q_{0.975}^{(B)}]$.

```
(bpi <- quantile(theta_boot, p=c(0.025, 0.975), names=FALSE))
```

```
[1] 1.726505 2.833912
```

The CBCI is then computed as $[2\hat{\theta} - Q_{0.975}^{(B)}, 2\hat{\theta} - Q_{0.025}^{(B)}]$.

```
(cbci <- 2*theta - rev(bpi))
```

```
[1] 1.438444 2.545851
```

Solution for e)

To obtain the CI based on predictive simulation follows a similar pattern as in c), with the difference that here we resample the response variable based on the full model

$$\begin{aligned} \text{art}_i &\sim \text{Negative Binomial}(\mu_i, \hat{\theta}) \\ \mu_i &= \exp(\hat{b}_0 + \hat{b}_1 \text{phd}_i + \hat{b}_2 \text{ment}_i). \end{aligned}$$

Again, during simulations, we set the parameter values to the ones estimated from the original data. For each simulation, we fit the full negative binomial regression model, and store the values of estimated parameter $\hat{b}_1^{(s)}$.

```
mu <- exp(b0 + b1 * bioChemists[["phd"]] + b2 * bioChemists[["ment"]])

S <- 1e3
b1_pred_sim <- numeric(S)
for (s in 1:S) {
  art_sim <- rnbino(N, theta, mu=mu)
  data_sim <- data.frame(
    art = art_sim,
    phd = bioChemists[["phd"]],
    ment = bioChemists[["ment"]]
  )
  fit_sim <- MASS::glm.nb(art ~ phd + ment, data = data_sim)
  b1_pred_sim[s] <- coefficients(fit_sim)["phd"]
}
```

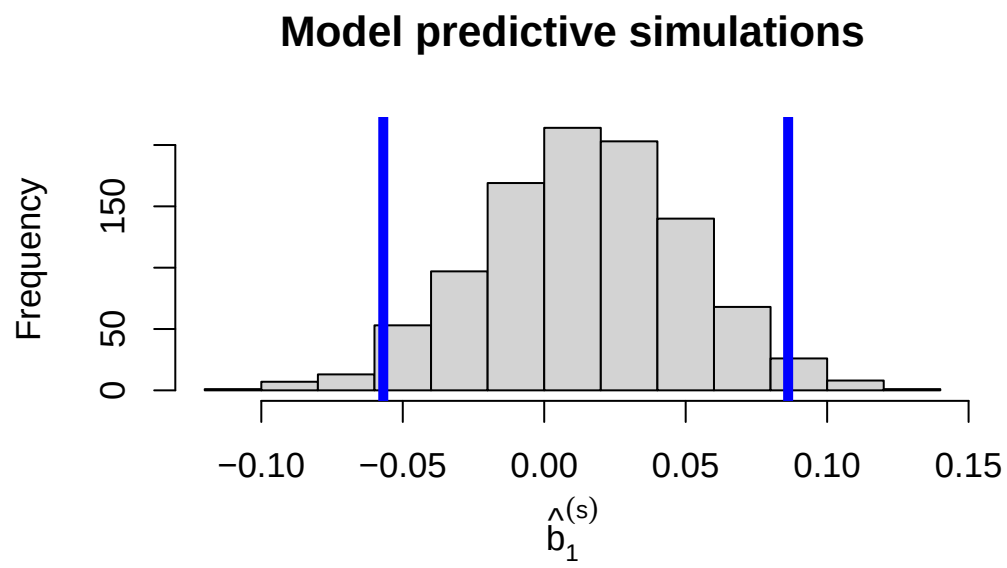
Compute the quantiles of the sampling distribution to obtain the CI.

```
(b1_pred_sim_ci <- quantile(b1_pred_sim, p=c(0.025, 0.975), names=FALSE))
```

```
[1] -0.05689043  0.08622123
```

We may also plot the entire sampling distribution with the CI overlayed (blue vertical lines).

```
par(cex=1.1)
hist(
  x=b1_pred_sim,
  main="Model predictive simulations",
  xlab=expression(hat(b)[1]^(s))
)
abline(v=b1_pred_sim_ci, col="blue", lwd=5)
```



Solution for f)

We define the two models as a Negative binomial regression but with different predictors,

$$\text{art}_i \sim \text{Negative Binomial}(\mu_i, \theta)$$

$$\mathcal{M}_0 : \mu_i = \exp(b_0 + b_2 \text{ment}_i)$$

$$\mathcal{M}_1 : \mu_i = \exp(b_0 + b_1 \text{phd}_i + b_2 \text{ment}_i)$$

```
models <- c(M_0 = art ~ ment, M_1 = art ~ phd + ment)
```

Now we compute the *in sample* log likelihood for each model.

```
loglik <- sapply(models, function(m)
  MASS::glm.nb(m, data = bioChemists) |> logLik()
)
loglik
```

```
      M_0      M_1
-1569.837 -1569.771
```

$\mathcal{M}_1$ has slightly higher log likelihood (fits the data a tiny bit better).

Next, we compute the *out of sample* log likelihood. Here we use a leave-one-out (loo), though you should be able to achieve similar results with alternatives (e.g., k-fold).

```
N <- nrow(bioChemists)
loo_loglik <- matrix(nrow = N, ncol=length(models))
colnames(loo_loglik) <- names(models)
for(i in 1:N) {
  train_data <- bioChemists[-i, ]
  test_data  <- bioChemists[ i, ]

  for(m in seq_along(models)) {
    fit <- MASS::glm.nb(models[[m]], data = train_data)
    y_hat <- predict(fit, newdata = test_data, type="response")
    theta <- fit[["theta"]]
    loo_loglik[i, m] <- dnbinom(
      test_data[["art"]], theta, mu = y_hat, log = TRUE
    )
  }
}

colSums(loo_loglik)
```

```
      M_0      M_1
-1573.354 -1574.558
```

Now $\mathcal{M}_0$ has higher out of sample log likelihood. $\mathcal{M}_1$ is more expressive than $\mathcal{M}_0$ due to the inclusion of an additional predictor `phd` and so is able to fit the data slightly better (in terms of in sample performance). However, the `phd` predictor is not very strong, and as a result, $\mathcal{M}_0$ outperforms $\mathcal{M}_1$ in predicting out of sample.